

RecruitMe — Project Scope, Current State & Running Costs

Prepared 2026-06-04; last updated 2026-06-15. Figures from live production data where measured; estimates flagged.

1. What it is

RecruitMe is a recruiter-facing SaaS that sources, scores, and manages candidates for open roles. A recruiter creates a **job** (pastes a JD → AI parses it into structured must-haves / nice-to-haves / seniority / location / salary), then **finds candidates** from three sources — an internal **talent pool** (~15k profiles), **LinkedIn**, and **SEEK Talent Search** — and the system **AI-scores** each candidate against the role, producing a ranked shortlist with a written rationale per candidate.

Multi-tenant (org-scoped), single primary operator today. Deployed on Railway; live discovery runs through a self-hosted mini-PC scraper.

2. Current state (what's built & working)

Scale (live, 2026-06-15): 15,317 candidates · 731 AI-scored · 14,662 with captured profiles · ~13.5k CVs stored (encrypted at rest) · 11 active jobs.

Tech stack

- **App:** Next.js 15 / React 19 / TypeScript, Prisma ORM, PostgreSQL 18 — hosted on **Railway** (auto-deploy from GitHub `main`).
- **AI:** Anthropic Claude — **Haiku 4.5** for parsing & snippet scoring (cheap), **Sonnet 4.6** for full-profile scoring (quality). Cost-tracked per call with a daily spend cap.
- **Scraper:** self-hosted mini-PC ("the box") running a persistent logged-in browser (Patchright) for LinkedIn + SEEK; reached over Tailscale. Single-worker by design (ban-avoidance).
- **Storage:** CVs in an S3-compatible bucket (`t3.storageapi.dev`), **AES-256-GCM encrypted at rest**, original sender format preserved.

Features shipped & working

- **JD analysis** — Claude parses a pasted JD into structured requirements + search expansion + title variants.
- **Multi-source candidate search** — internal pool (Postgres full-text + AI ranking), live LinkedIn scrape, live SEEK Talent Search scrape. Library-first (cheap pool results instant; live sources on demand).
- **AI scoring** — per-candidate match score (0–100) with must-have coverage, category breakdown, and a recruiter summary; provisional (snippet) vs full-profile scoring; per-org configurable weights; cached so unchanged candidates aren't re-billed.
- **SEEK Talent Search** — now ingests result cards as candidates (credit-safe: harvests free card data, no per-profile credit spend). *Validated 2026-06-04.*
- **LinkedIn discovery** — people-search → profiles scraped → ingested + scored. *(Fixed 2026-06-15: a stale hardcoded NZ geo-filter parameter that LinkedIn had silently stopped honouring was making every LinkedIn search return "No results found" — verified on the box, then removed. Results are NZ-biased via the logged-in account; the app re-narrows by the requested location.)*
- **CRM — clients / submissions / placements** (built, flag-gated behind `FEATURES_CRM_ENABLED` pending enablement) — client records, "submit candidate to client" from a job, placement tracking with fee/guarantee/invoice + auto-reminders. Finishes the previously half-built CRM; all create paths org-FK-validated.
- **Talent pool / flywheel** — every search enriches the pool; future searches serve those candidates instantly.
- **Candidate management** — library, per-job pipelines (shortlist/contacted/etc.), CV upload + text extraction, identity dedup across sources (LinkedIn / SEEK / JobAdder), screening data, recruiter notes, CSV export.
- **CV handling** — extraction from PDF/Word/RTE, encryption at rest, original-format preservation.
- **Ops** — on-box health dashboard (CPU/RAM/disk/temp, scraper status), cost attribution + daily spend cap, PII redaction on the seller heartbeat, CI real-DB test gate, error reporting.
- **Security & data-safety floor** (shipped 2026-06-15) — CV encryption now fails closed (refuses to mint a new key when encrypted CVs exist, so a missing key can't silently orphan them); cross-org candidate-tag read/write leak closed; four AI-cost-attribution gaps closed (spend now always billed to the right org); Sentry PII scrubber so candidate names/emails/profile text don't leave the app; Llama-offload default flipped off (the box can't score, so a credit-out now fails cleanly instead of hanging).
- **Live JobAdder feed** — the operator's JobAdder candidate data (~13k profiles) is available live and on demand, drawn from the operator's own JobAdder account, and kept current in the platform.

Known constraints / debt

- **Two parallel search systems** — now largely converged onto one durable, resumable multi-source run (job-scoped, live-streaming). The legacy job-search path is mostly retired; final dead-code cleanup is the remaining item — see roadmap §3.
 - SEEK candidates are **snippet-level** (name/role/company/location); full-profile enrichment is a separate, credit-gated action (scraper exists, not auto-fired).
 - ~15k candidates, **731 scored** — large unscored backlog (scoring is on-demand + cost-capped).
 - Single scraper worker (no redundancy); the box must stay online for live discovery.
 - Scraper still authenticates via a shared secret (org-binding token cutover incomplete).
-

3. Future implementations (roadmap)

The authoritative, sequenced plan to take the app to feature-complete + self-running lives in `docs/completion-plan.md` (7 phases, built from a verified 6-dimension audit). The items below are the strategic summary.

A. Search architecture consolidation (*planned, documented in `docs/search-architecture-plan.md`*) — collapse the two parallel search systems into one **federated dispatcher + pluggable source adapters** (pool/LinkedIn/SEEK/+future), one durable result model, and a **live-streaming results UI** (results appear as they're scraped). Makes adding a new source trivial and removes the current fragility. ~6 phases, each independently shippable.

B. SEEK full-profile enrichment — one-click, credit-aware deep-fetch of a chosen SEEK candidate's full profile (the scraper exists; needs wiring + a credit guard).

C. Scoring at scale — score the unscored backlog; background/batch scoring; richer re-score-on-change.

D. Talent flywheel — later phases — background discovery that continuously enriches the pool (cost-gated); profile freshness / re-fetch of stale profiles.

E. Candidate UX polish — shipped (email/screening/file-count surfacing, import-batch grouping, source badges). Remaining: candidate-card density declutter.

F. Hardening — complete the scraper org-binding token cutover; multi-worker / redundancy if scrape volume grows; observability.

Priority order is the operator's call; A unblocks the most future work.

4. Running costs per month

Item	Cost / month	Basis
Claude API (Anthropic)	~\$15–40	Measured \$13.37 over the last 30 days (865 calls, ~3.7M tokens) — but that was mostly cheap Haiku. With Sonnet now scoring full profiles (~\$0.05/score) cost rises with scoring volume; hard-capped at \$5/day (~ \$150/mo ceiling) by the spend guard. Realistic steady-state ~\$20–40.
Railway (app + Postgres + egress)	\$25	Hosting: always-on Next app + Postgres (5 GB volume) + bandwidth. Operator-confirmed figure.
CV storage (S3 / t3.storageapi.dev)	~\$1–3	~13.5k encrypted CVs (~8–12 GB est.) at S3-compatible rates + light egress.
Mini-PC scraper (electricity)	~\$3–6	Low-power i3 mini-PC 24/7 at NZ power rates. Internet = existing home line; Tailscale = free tier.
Domain	\$0	Railway subdomain (<code>*.up.railway.app</code>). Custom domain would add ~\$1–2/mo.
Platform infra subtotal	~\$45–75 / month	Everything RecruitMe itself costs to run (recurring).
Mini-PC hardware (one-off)	\$150 once	The mini-PC itself — already purchased; one-time capital cost, not recurring (≈ \$4–6/mo amortised over 2–3 yrs).
SEEK Talent Search	<i>operator's existing subscription</i>	External recruiting tool (the box logs into the operator's SEEK account). Significant but pre-existing; per-search/profile credits apply — the app is built to be credit-safe.
LinkedIn	<i>operator's existing account</i>	Scraper uses the operator's logged-in LinkedIn; no extra app cost.

Bottom line: the RecruitMe *platform* runs at roughly **\$45–75/month** in direct infrastructure + AI cost (plus a one-off **\$150** mini-PC, already purchased), on top of the operator's existing SEEK/LinkedIn/JobAdder subscriptions. The single largest and most variable line is the Claude API, which is bounded by the daily spend cap and scales with how much scoring is done.

5. Tech-stack summary (for a developer handoff)

- **Frontend/Backend:** Next.js 15 (App Router), React 19, TypeScript, Tailwind.
- **Data:** PostgreSQL 18 + Prisma; Postgres full-text search (FTS) for the talent pool.
- **AI:** Anthropic SDK (Claude Haiku 4.5 + Sonnet 4.6); structured scoring pipeline with caching + cost tracking + spend caps.
- **Scraping:** Node/TypeScript worker on a mini-PC, Patchright (patched Chromium), one persistent session per platform; polls the app's job queue (Postgres-backed `ScrapeJob` , priority + dedup).
- **Infra:** Railway (CI real-DB gate on deploy), Tailscale (box networking), S3-compatible blob storage (encrypted CVs).
- **Repo:** GitHub `baeley-canning/RecruitMe` , deploy = push to `main` .